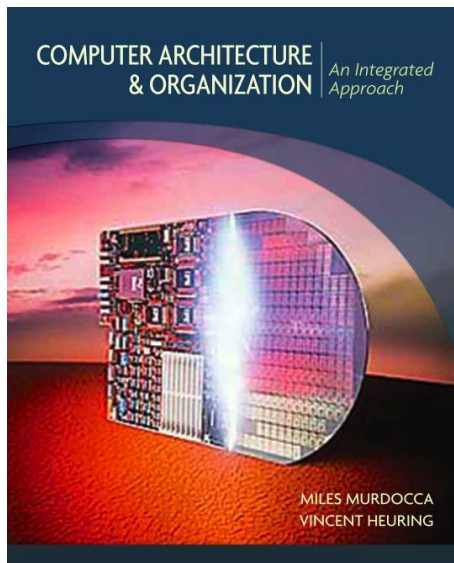


Organización de las Computadoras

Carolina Chaves Garcia



Programación directa (3)

Manejo de memoria, arreglos y cadenas

Manejo de memoria, arreglos y cadenas

Manejo de Memoria en RISC-V

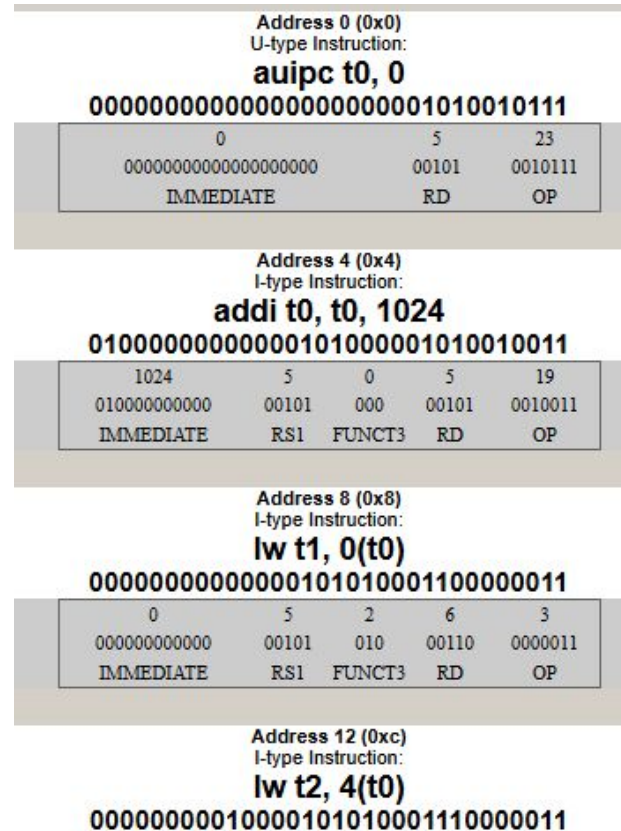
- La memoria en RISC-V es byte-addressable, es decir, cada byte tiene una dirección única.
- Las palabras se alinean a direcciones múltiplos de 4.
- Los registros son rápidos pero limitados, la memoria es lenta pero abundante.

Dec. Val. (dword)	Dec. Val. (word)	Byte 3 (dec.val.)	Byte 2 (dec.val.)	Byte 1 (dec.val.)	Byte 0 (dec.val.)	Addr.
0	0	00000000 (0)	00000000 (0)	00000000 (0)	00000000 (0)	1024
	0	00000000 (0)	00000000 (0)	00000000 (0)	00000000 (0)	1028
0	0	00000000 (0)	00000000 (0)	00000000 (0)	00000000 (0)	1032
	0	00000000 (0)	00000000 (0)	00000000 (0)	00000000 (0)	1036
0	0	00000000 (0)	00000000 (0)	00000000 (0)	00000000 (0)	1040
	0	00000000 (0)	00000000 (0)	00000000 (0)	00000000 (0)	1044

Manejo de memoria, arreglos y cadenas

- Instrucciones de acceso a memoria:

- lw (load word): carga una palabra desde memoria a registro
- sw (store word): almacena una palabra desde registro a memoria
- lb/sb (load/store byte): para trabajar con bytes individuales
- lh/sh (load/store halfword): para medios palabras (16 bits)



Ejm 1

Ejm 1:

```
addi a0,x0,5
addi a1,x0,12
addi t0,x0,3
addi sp, sp, -12      # Reserva 12 bytes para 3 variables (0, 4, 8)
sw a0, 0(sp)          # Guarda a0 en offset 0
sw a1, 4(sp)          # Guarda a1 en offset 4
lw t0, 8(sp)          # Lee variable local (offset positivo)
addi sp, sp, 12       # Libera el espacio del stack
```

Manejo de memoria, arreglos y cadenas

Ejm 1

```
addi a0,x0,5
addi a1,x0,12
addi t0,x0,3
```

5	t0	3	00000000000000000000000000000011
10	a0	5	00000000000000000000000000000101
11	a1	12	000000000000000000000000000001100
2	sp	5120	00000000000000000000101000000000

```
addi sp, sp, -12    # Reserva 12 bytes para 3 variables (0, 4, 8)
```

0	00000000 (0)	00000000 (0)	00000000 (0)	00000000 (0)	5100
0	00000000 (0)	00000000 (0)	00000000 (0)	00000000 (0)	5104
0	00000000 (0)	00000000 (0)	00000000 (0)	00000000 (0)	5108
0	00000000 (0)	00000000 (0)	00000000 (0)	00000000 (0)	5112
0	00000000 (0)	00000000 (0)	00000000 (0)	00000000 (0)	5116

Manejo de memoria, arreglos y cadenas

Ejm 1

sw a0, 0(sp) # Guarda a0 en offset 0

0	00000000 (0)	00000000 (0)	00000000 (0)	00000000 (0)	5104
5	00000000 (0)	00000000 (0)	00000000 (0)	00000101 (5)	5108
0	00000000 (0)	00000000 (0)	00000000 (0)	00000000 (0)	5112
0	00000000 (0)	00000000 (0)	00000000 (0)	00000000 (0)	5116

sw a1, 4(sp) # Guarda a1 en offset 4

5	00000000 (0)	00000000 (0)	00000000 (0)	00000101 (5)	5108
12	00000000 (0)	00000000 (0)	00000000 (0)	00001100 (12)	5112
0	00000000 (0)	00000000 (0)	00000000 (0)	00000000 (0)	5116

Manejo de memoria, arreglos y cadenas

Ejm 1

`lw t0, 8(sp)` # Lee variable local (offset positivo)

1	ra	0	00000000000000000000000000000000
2	sp	5108	0000000000000000000000001001111110100
3	gp	1024	000000000000000000000000100000000000
4	tp	0	0000000000000000000000000000000000
5	t0	0	0000000000000000000000000000000000
6	t1	0	0000000000000000000000000000000000
7	t2	0	0000000000000000000000000000000000
8	s0/fp	5120	000000000000000000000000101000000000
9	s1	0	0000000000000000000000000000000000
10	a0	5	000000000000000000000000000000000101
11	a1	12	000000000000000000000000000000001100

Ejm 2

Ejm2:

```
# Almacenar una palabra en memoria
sw t2, 4(t1)      # memoria[t1 + 4] = t2
# Cargar una palabra desde memoria
lw t0, 0(t1)      # t0 = memoria[t1 + 0]

.data
persona:
    .word 25      # edad (offset 0)
    .word 180     # altura (offset 4)
    .word 75000   # salario (offset 8)

.text
la t0, persona

lw t1, 0(t0)      # Lee edad (offset 0)
lw t2, 4(t0)      # Lee altura (offset 4)
lw t3, 8(t0)      # Lee salario (offset 8)
```

.data

```
.word 25      # edad (offset 0)
.word 180     # altura (offset 4)
.word 75000   # salario (offset 8)
```

```
# Cargar una palabra desde memoria
```

5	t0	1024	000000000000000000000000100000000000
---	----	------	--------------------------------------

Manejo de memoria, arreglos y cadenas

Ejm 2

```
lw t1, 0(t0)      # Lee edad (offset 0)
lw t2, 4(t0)      # Lee altura (offset 4)
lw t3, 8(t0)      # Lee salario (offset 8)
```

5	t0	1024	0000000000000000000000001000000000
6	t1	25	0000000000000000000000000000000011001
7	t2	180	0000000000000000000000000000000010110100
28	t3	75000	00000000000000000000000010010010011111000

Manejo de memoria, arreglos y cadenas

Arreglos en Ensamblador

- Los arreglos son bloques contiguos de memoria
- Para acceder a un elemento: $\text{dirección_base} + \text{índice} \times \text{tamaño_elemento}$
- Ejm:

```
.data
```

```
arreglo: .word 10, 20, 30, 40, 50    # 5 elementos de 4 bytes cada uno
```

```
.text
```

```
# Acceder al tercer elemento (índice 2)
```

```
la t0, arreglo    # t0 = dirección base del arreglo
```

```
li t1, 2          # t1 = índice deseado
```

```
slli t2, t1, 2    # t2 = índice × 4 (desplazamiento en bytes)
```

```
add t3, t0, t2    # t3 = dirección del elemento
```

```
lw a0, 0(t3)     # a0 = valor del tercer elemento (30)
```

Manejo de memoria, arreglos y cadenas

Arreglos en Ensamblador

arreglo: .word 10, 20, 30, 40, 50 # 5 elementos de 4 bytes cada uno

Dec. Val. (dword)	Dec. Val. (word)	Byte 3 (dec.val.)	Byte 2 (dec.val.)	Byte 1 (dec.val.)	Byte 0 (dec.val.)	Addr.
8589934 5930	10	00000000 (0)	00000000 (0)	00000000 (0)	00001010 (10)	1024
	20	00000000 (0)	00000000 (0)	00000000 (0)	00010100 (20)	1028
1717986 91870	30	00000000 (0)	00000000 (0)	00000000 (0)	00011110 (30)	1032
	40	00000000 (0)	00000000 (0)	00000000 (0)	00101000 (40)	1036
50	50	00000000 (0)	00000000 (0)	00000000 (0)	00110010 (50)	1040
	0	00000000 (0)	00000000 (0)	00000000 (0)	00000000 (0)	1044

Manejo de memoria, arreglos y cadenas

Arreglos en Ensamblador

```
la t0, arreglo      # t0 = dirección base del arreglo
li t1, 2             # t1 = índice deseado
```

5	t0	1024	00000000000000000000000000000000 0000000000000000000000001000000000
6	t1	2	00000000000000000000000000000000 00000000000000000000000000000010

```
slli t2, t1, 2      # t2 = índice × 4 (desplazamiento en bytes)
```

6	t1	2	00000000000000000000000000000000 00000000000000000000000000000010
7	t2	8	00000000000000000000000000000000 00000000000000000000000000001000

Manejo de memoria, arreglos y cadenas

Arreglos en Ensamblador

```
add t3, t0, t2      # t3 = dirección del elemento
```

Dec. Val. (dword)	Dec. Val. (word)	Byte 3 (dec.val.)	Byte 2 (dec.val.)	Byte 1 (dec.val.)	Byte 0 (dec.val.)	Addr.
8589934 5930	10	00000000 (0)	00000000 (0)	00000000 (0)	00001010 (10)	1024
	20	00000000 (0)	00000000 (0)	00000000 (0)	00010100 (20)	1028
1717986 91870	30	00000000 (0)	00000000 (0)	00000000 (0)	00011110 (30)	1032
	40	00000000 (0)	00000000 (0)	00000000 (0)	00101000 (40)	1036
50	50	00000000 (0)	00000000 (0)	00000000 (0)	00110010 (50)	1040

```
lw a0, 0(t3)      # a0 = valor del tercer elemento (30)
```

10	a0	30	00000000000000000000000000000000 00000000000000000000000000000000
----	----	----	--

Cadenas de Caracteres

- Los arreglos son bloques contiguos de memoria
- Para acceder a un elemento: $\text{dirección_base} + \text{índice} \times \text{tamaño_elemento}$

```
.data
mensaje: .string "Hola Mundo!"    # Cadena terminada en null

.text
# Recorrer una cadena
la t0, mensaje
loop:
    lb t1, 0(t0)          # Cargar un carácter
    beq t1, zero, fin      # Si es cero, terminar
    # ... procesar carácter ...
    addi t0, t0, 1         # Avanzar al siguiente carácter
    j loop
fin:
```

Manejo de memoria, arreglos y cadenas

Cadenas de Caracteres

```
.data
```

```
mensaje: .string "Hola Mundo!"    # Cadena terminada en null
```

Dec. Val. (word)	Byte 3 (dec.val.)	Byte 2 (dec.val.)	Byte 1 (dec.val.)	Byte 0 (dec.val.)	Addr.
16344963 28	01100001 (97)	01101100 (108)	01101111 (111)	01001000 (72)	1024
18531812 16	01101110 (110)	01110101 (117)	01001101 (77)	00100000 (32)	1028
2191204 (0)	00000000 (0)	00100001 (33)	01101111 (111)	01100100 (100)	1032

```
la t0, mensaje
```

5	t0	1024	000000000000000000000000010000000000
---	----	------	--------------------------------------

Cadenas de Caracteres

```
.text
# Recorrer una cadena
la t0, mensaje
loop:
    lb t1, 0(t0)          # Cargar un carácter
    beq t1, zero, fin      # Si es cero, terminar
    # ... procesar carácter ...
    addi t0, t0, 1         # Avanzar al siguiente carácter
    j loop
```

[illegible]

Manejo de memoria, arreglos y cadenas

Cadenas de Caracteres

● ● ● ● ● ● ● ● ●

5	t0	1033	0000000000000000000010000001001
6	t1	111	000000000000000000000000000001101111
5	t0	1034	00000000000000000000000010000001010
6	t1	33	0000000000000000000000000000000100001
5	t0	1035	00000000000000000000000010000001011
6	t1	0	000000000000000000000000000000000000

```
fin:
```

5	t0	1035	0000000000000000000010000001011
6	t1	0	00000000000000000000000000000000

Manejo de memoria, arreglos y cadenas

Cadenas de Caracteres

FULL LOOPS		CPU Cycles																																																																																						
Instruction	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87																																					
beq t1, x0, 24	M	W																																																																																						
addi t0, t0, 1	X	M	W																																																																																					
jal x0, -24	D	X	M	W																																																																																				
lb t1, 0(t0)		F	D	X	M	W																																																																																		
beq t1, x0, 24			F	-	-	D	X	M	W																																																																															
addi t0, t0, 1					F	D	X	M	W																																																																															
jal x0, -24						F	D	X	M	W																																																																														
lb t1, 0(t0)							F	D	X	M	W																																																																													
beq t1, x0, 24								F	-	-	D	X	M	W																																																																										
addi t0, t0, 1											F	D	X	M	W																																																																									
jal x0, -24												F	D	X	M	W																																																																								
lb t1, 0(t0)													F	D	X	M	W																																																																							
beq t1, x0, 24														F	-	-	D	X	M	W																																																																				
addi t0, t0, 1															F	D	X	M	W																																																																					
jal x0, -24																F	D	X	M	W																																																																				
lb t1, 0(t0)																	F	D	X	M	W																																																																			
beq t1, x0, 24																		F	-	-	D	X	M	W																																																																
addi t0, t0, 1																			F	D	X	M	W																																																																	
jal x0, -24																				F	D	X	M	W																																																																
lb t1, 0(t0)																					F	D	X	M	W																																																															
beq t1, x0, 24																						F	-	-	D	X	M	W																																																												
addi t0, t0, 1																							F	D	X	M	W																																																													
jal x0, -24																								F	D	X	M	W																																																												
lb t1, 0(t0)																									F	D	X	M	W																																																											
beq t1, x0, 24																										F	-	-	D	X	M	W																																																								
addi t0, t0, 1																											F	D	X	M	W																																																									
jal x0, -24																												F	D	X	M	W																																																								
lb t1, 0(t0)																													F	D	X	M	W																																																							
beq t1, x0, 24																														F	-	-	D	X	M	W																																																				
addi t0, t0, 1																															F	D	X	M	W																																																					
jal x0, -24																																F	D	X	M	W																																																				
lb t1, 0(t0)																																	F	D	X	M	W																																																			
beq t1, x0, 24																																		F	-	-	D	X	M	W																																																
addi t0, t0, 1																																			F																																																					

Registros de estado y control del procesador

Registros de estado y control del procesador

Introducción

- Los registros de estado y control (CSR) son un conjunto de registros (4096) con direcciones de memoria independientes de los registros de propósito general.
- Nos **ayudan a almacenar y gestionar información** de la CPU:
 - Configuración del sistema
 - Estado de ejecución
- Aunque están definidos en un espacio separado, la **mayoría** de los CSR están **restringidos** por protección. Solo podemos acceder a ciertos CSR para la medición de rendimiento y para el manejo de punto flotante.
- Almacenan información esencial sobre el estado del procesador y su configuración.
- Relación con los Registros de Propósito General
 - Los 32 registros de propósito general de 32 bits (incluido el registro cero) son usados por la unidad central de procesamiento para sus operaciones aritméticas, lógicas y de acceso a memoria. Son el “área de trabajo” para las instrucciones.
 - Los CSRs
 - Procesan directamente los datos de la ALU
 - Actúan como un centro de control y estado para la unidad que gestiona el procesador.
 - Son registros dedicados a la administración y el control del estado interno del procesador RISC-V

Introducción

- **mstatus (Machine Status Register)**
 - Contiene información global del estado de la máquina
 - Bits importantes:
 - MIE: Machine Interrupt Enable (habilitar interrupciones)
 - MPIE: Previous MIE (valor anterior de MIE)
 - MPP: Previous Privilege Mode (modo de privilegio anterior)
- **mie (Machine Interrupt Enable Register)**
 - Controla qué interrupciones están habilitadas
 - Bits por tipo de interrupción: MSIE, MTIE, MEIE
- **mip (Machine Interrupt Pending Register)**
 - Indica qué interrupciones están pendientes

Instrucciones para Manipular CSR

Leer un registro de control:

```
csrrw t0, mstatus, x0    # t0 = mstatus
```

Escribir un registro de control:

```
csrrw x0, mstatus, t0    # mstatus = t0
```

Operaciones atómicas (leer-modificar-escribir):

```
csrrs t1, mstatus, t2    # t1 = mstatus; mstatus = mstatus | t2
```

```
csrrc t1, mstatus, t2    # t1 = mstatus; mstatus = mstatus & ~t2
```

Registros de estado y control del procesador

Flags y Saltos Condicionales

Comparar registros

slt t0, t1, t2 # t0 = 1 si t1 < t2, 0 en otro caso

sltu t0, t1, t2 # versión sin signo

Saltos condicionales basados en comparaciones

blt t1, t2, etiqueta1 # salta si t1 < t2

bge t1, t2, etiqueta2 # salta si t1 >= t2

beq t1, t2, etiqueta3 # salta si t1 == t2

bne t1, t2, etiqueta4 # salta si t1 != t2

etiqueta1:

addi a0,zero,1

etiqueta2:

addi a0,zero,2

etiqueta3:

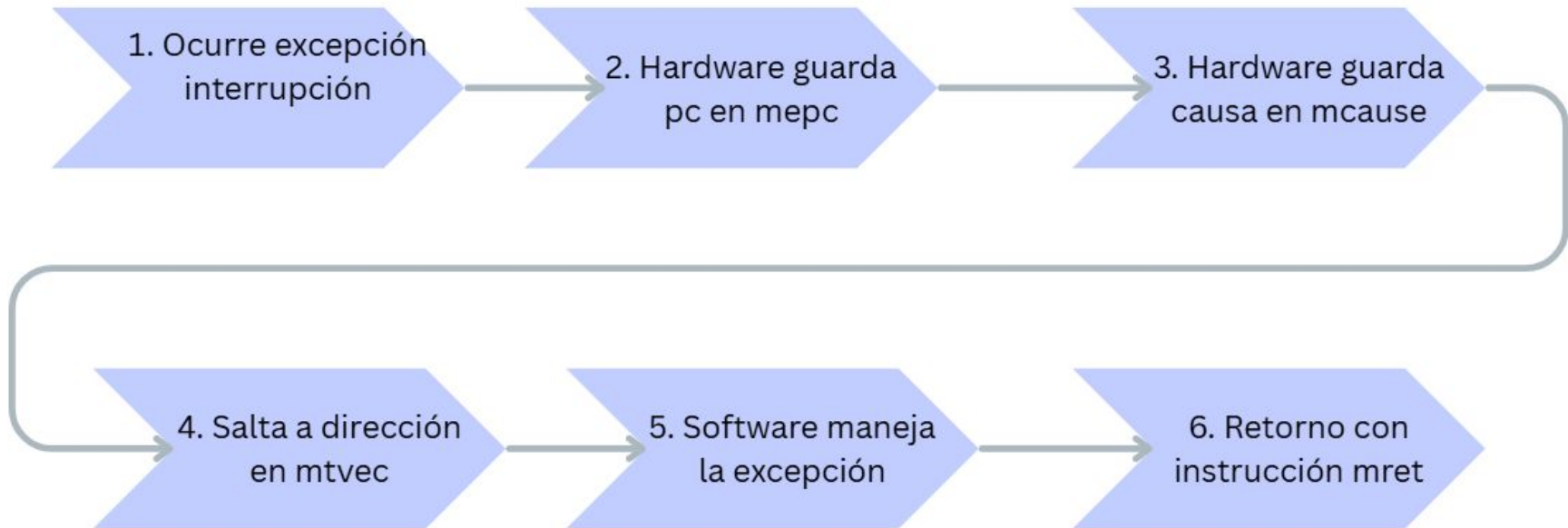
addi a0,zero,3

etiqueta4:

addi a0,zero,4

Registros de estado y control del procesador

Manejo de Excepciones e Interrupciones



Llamadas al sistema y acceso a hardware

Llamadas al sistema y acceso a hardware

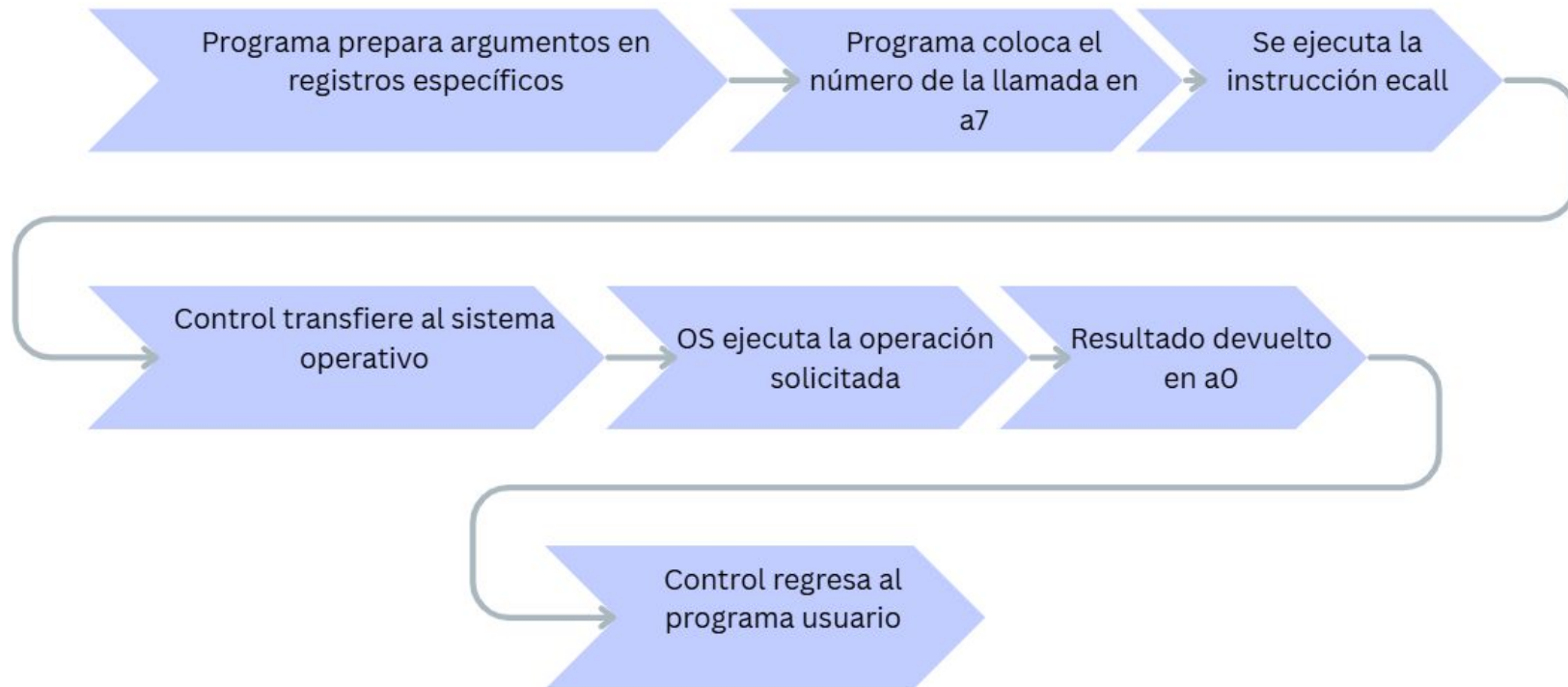
Introducción

- Las llamadas al sistema son la **interfaz** entre **programas** y el **sistema operativo (OS)**, por lo tanto implica el uso de instrucciones específicas para interactuar con el hardware.
- **ECALL (environment call):**
 - La usamos para una llamada al sistema.
 - El control se transfiere al sistema operativo permitiendo que el programa solicite servicios de hardware o del OS.
- **EBREAK (environment break):**
 - Se usa para depuración, permitiéndole al programa detenerse en puntos específicos.
- Esto nos permite un acceso controlado a recursos de hardware.
- A través de los registros las llamadas al sistema pasan parámetros y reciben resultados.

Llamadas al sistema y acceso a hardware

Mecanismo de Llamadas al Sistema en RISC-V

Flujo de un system call



Registros usados para system calls:

- a7: Número de la llamada al sistema
- a0-a2: Argumentos (dependiendo de la llamada)
- a0: Valor de retorno

Llamadas al sistema y acceso a hardware

Niveles de Privilegio en RISC-V

- Modos de ejecución:
 - User (U): Nivel menos privilegiado, para aplicaciones
 - Supervisor (S): Para sistemas operativos
 - Machine (M): Máximo privilegio, para firmware
- Transiciones:
 - ecall eleva privilegios de $U \rightarrow S$ o $S \rightarrow M$
 - mret/sret retornan a modo menos privilegiado

Llamadas al sistema y acceso a hardware

Memory-Mapped I/O (Acceso a Hardware)

- Dispositivos de hardware se mapean a direcciones de memoria
- Leer/escribir estas direcciones controla el hardware
- Instrucción ECALL vs Acceso Directo

ECALL	Acceso Directo
Mediado por SO	Acceso directo
Seguro	Potencialmente peligroso
Portable	Específico de hardware
Más lento	Más rápido

Técnicas de optimización en ensamblador

Técnicas de optimización en ensamblador

Introducción

- La optimización implica **minimizar accesos a memoria, utilizar instrucciones eficientes, y aprovechar la segmentación del procesador.**
- Asegurarnos que:
 - **Solo** las instrucciones de carga (load) y almacenamiento (store) accedan a la memoria
 - Las operaciones aritméticas se realizan sólo en los registros de la CPU.
- Optimización basada en el hardware:
 - Segmentación (Pipelining): Estructurar el código de manera que la segmentación sea lo más eficiente posible.
 - Evitar dependencias de datos entre instrucciones para mantener el pipeline de la CPU funcionando de manera fluida, evitando que se detenga.

- Optimizaciones de rendimiento:
 - Reducir ciclos de reloj
 - Minimizar accesos a memoria
 - Maximizar uso de registros
- Optimizaciones de tamaño:
 - Reducir instrucciones
 - Compactar datos
 - Reutilizar código

Técnicas de optimización en ensamblador

Técnicas Específicas de RISC-V

- **Uso eficiente de registros:**

- Ineficiente

```
.data
valor1: .word 20
valor2: .word 30
resultado: .word 0
```

```
.text
la t3, valor1
lw t0, 0(t3)
la t3, valor2
lw t1, 0(t3)
add t2, t0, t1
la t3, resultado
sw t2, 0(t3)
```

Muchos accesos a memoria

- Eficiente

```
.data
resultado: .word 0      # Solo si necesitas
guardar en memoria
```

```
.text
# Los valor1 ya están en registros s0 y
s1(Esto podría venir de cálculos
previos, parámetros, etc.)
add a0, s0, s1          # Resultado en
registro de salida a0
# Solo si necesitas guardar en memoria:
la t0, resultado
sw a0, 0(t0)            # Guardar resultado
en memoria
```

Mantener valores en registros

Técnicas de optimización en ensamblador

Técnicas Específicas de RISC-V

- Instrucciones de desplazamiento y extensión:

- Multiplicar por constante potencia de 2

`slli t0, t1, 3 # t0 = t1 * 8 (mejor que mul t0, t1, 8)`

- Instrucciones de inmediato:

- Usar immediatos cuando sea posible

`addi t0, t1, 42 # Mejor que li t2, 42; add t0, t1, t2`

- Optimización de Bucles

- Transformaciones comunes:
- Desenrollado de bucles (loop unrolling)
- Fusión de bucles (loop fusion)
- Intercambio de bucles anidados (loop interchange)

Técnicas de optimización en ensamblador

Técnicas Específicas de RISC-V

- Alineación de Memoria y Accesos:

- Asegurar alineación para accesos eficientes

```
.align 3          # Alinear a 8 bytes
datos: .dword 1, 2, 3, 4
```

- Branch Prediction y Optimización de Saltos:

- Colocar el camino más probable primero

```
bge t0, t1, camino_raro # Asumiendo que t0 < t1 es más
común
# Código para camino común
j fin
camino_raro:
# Código para caso raro
fin:
```


Uso de herramientas de depuración

Uso de herramientas de depuración

Introducción

- Utilizamos herramientas conocidas como depuradores, las cuales interactúan con un simulador o dispositivo de hardware para controlar la ejecución del código.
- De esta forma podemos
 - Hacer un seguimiento detallado
 - Establecer puntos de interrupción
 - Examinar el estado de los registros y la memoria
- Cómo Funciona la Depuración
 1. Conexión: El depurador se conecta al sistema objetivo (simulador o hardware).
 2. Puntos de interrupción (Breakpoints): Se establecen en líneas específicas del código para detener la ejecución y examinar el estado del programa en ese momento.
 3. Paso a paso (Stepping): Se puede ejecutar el código instrucción por instrucción para entender el flujo de ejecución.
 4. Inspección de registros: Se pueden visualizar los valores de los registros de la CPU para entender el estado interno del procesador.
 5. Inspección de memoria: Se pueden ver y modificar el contenido de la memoria.

Uso de herramientas de depuración

Herramientas Comunes para RISC-V

- GDB (GNU Debugger)
 - Depurador estándar para múltiples arquitecturas
 - Soporte para RISC-V a través de extensions
 - Interfaz de línea de comandos y visuales disponibles
- Simuladores:
 - Spike: Simulador de referencia de RISC-V
 - QEMU: Emulador con soporte para debugging
 - RARS: RISC-V Assembler and Runtime Simulator
- Herramientas específicas:
 - OpenOCD: Para debugging sobre hardware real
 - Tracer: Para generar trazas de ejecución

Uso de herramientas de depuración

Errores comunes

Error	Síntomas	Solución
Registros no preservados	Comportamiento errático	Guardar/restaurar registros s*
Acceso desalineado	Excepción de carga/almacen	Asegurar alineación
Desbordamiento de pila	Corrupción de memoria	Verificar gestión de sp
Saltos incorrectos	Comportamiento impredecible	Verificar direcciones

Gracias